

Explicación Detallada – Labs 07 y 08 (CS 25000) – En Español

LAB 07 – Experimentos y Preguntas 2

Este laboratorio enseña cómo se genera un programa ejecutable en C a través de las etapas del compilador: preprocesador, compilador, ensamblador y linker. Cada etapa transforma el código fuente hasta producir un archivo ejecutable.

Experimento 1 – El Preprocesador

El preprocesador toma el archivo fuente `hello.c` y genera `hello.i`. En `hello.i` se eliminan las directivas `#define`, los comentarios y las líneas `#include`. El compilador sustituye las macros y valores constantes, moviendo trabajo del tiempo de ejecución al tiempo de compilación para hacerlo más eficiente.

Experimento 2 – El Compilador

Si intentas compilar `hello.c` renombrado como `hello.i`, el compilador muestra errores, porque detecta símbolos no válidos (`#`, `<`, `>`). Cuando se vuelve a generar con `gcc -E -P hello.c -o hello.i`, `hello.i` contiene ya un programa en C puro. Después, `gcc -S hello.i -o hello.s` produce el código ensamblador. El sistema usa arquitectura AArch64, creada por Arm Holdings.

Experimento 3 – El Ensamblador

El ensamblador convierte `hello.s` en `hello.o` (binario). Al ejecutar `nm hello.o`, vemos símbolos como `main` (definido) y `printf`, `puts` (no definidos). El tipo de símbolo `T` indica que pertenece a la sección de texto del programa. Las funciones no definidas serán resueltas luego por el linker.

Experimento 4 – El Linker

El linker combina `hello.o` con las librerías estáticas, asigna direcciones a cada símbolo y produce el ejecutable `hello`. Por ejemplo, el símbolo `main` tiene una dirección específica (`0x7d8`) y antes de él hay 2008 bytes de información. Las funciones externas (como `printf` y `puts`) son enlazadas en tiempo de carga por el loader.

Conclusión del Lab 07: Aprendemos cómo un programa en C pasa de texto fuente a ejecutable binario. Cada etapa cumple una función distinta: preprocesador (expande macros), compilador (traduce a ensamblador), ensamblador (traduce a código máquina) y linker (une todo en un ejecutable).

LAB 08 – Dependencia de Datos

Este laboratorio trata sobre dependencias en instrucciones y convenciones de registros en arquitectura LEGv8.

1. Procesadores Pipelined

Los chips modernos usan pipelines para ejecutar varias instrucciones en paralelo, mejorando el rendimiento. Los programadores de aplicaciones no deben preocuparse directamente, pero los compiladores y diseñadores de sistemas sí deben considerar riesgos de dependencias (hazards).

2. Idea de Arquitectura

La “gran idea” es *Hacer rápido el caso común*: tener suficientes registros para pasar argumentos evita

accesos innecesarios a memoria.

3. Registros no preservados

Registros X0–X15 no se preservan porque el llamador puede sobreescribirlos; se usan para argumentos, resultados o temporales.

4. Uso de X20

Como X20 es callee-saved, la subrutina debe guardarlo en la pila al iniciar y restaurarlo al salir: STUR X20, [SP, #-16]! LDUR X20, [SP], #16

5. Dependencias en SUB

SUB X9, X7, X8 SUB X8, X9, X5 Aquí hay una dependencia verdadera en X7 y una anti-dependencia en X8.

6. Dependencias entre instrucciones

Las dependencias RAW, WAR y WAW indican cuándo una instrucción depende del resultado o uso de otra. Se puede deducir un posible orden: ADD X4, X1, X2 AND X6, X1, X3 SUB X5, X4, X6 XOR X1, X6, X4

7. Inserción de NOP

Si el hardware necesita 1 ciclo para pasar datos, se insertan NOPs: ADD X1, X2, X3 NOP AND X4, X1, X6 NOP SUB X5, X1, X8

8. Reordenamiento

No se puede eliminar los NOPs porque las tres instrucciones dependen del mismo registro X1.

9. Dependencia incierta en ejecución

LDUR X1, [X2] STUR X3, [X2] La dependencia verdadera (RAW) depende de si ambas direcciones apuntan al mismo lugar en tiempo de ejecución.

10. Eliminación de dependencias

Solo las WAR y WAW pueden eliminarse asignando registros distintos; las RAW son verdaderas dependencias de datos.

Conclusión del Lab 08: Las dependencias de datos afectan el orden de ejecución y el rendimiento en pipelines. Entenderlas permite optimizar código y evitar stalls o esperas innecesarias.